✈ SkyDesk

Flight Booking System - OOP Lab Sheet

Java • JavaFX • Intermediate Level

ℹ Learning Objectives - by the end of this lab you will be able to:

- Design and implement an inheritance hierarchy using an abstract class
- Apply encapsulation with private fields, validated getters and setters
- Use polymorphism to write code that works with any Seat subtype
- Implement a Java interface (Payable) in multiple unrelated classes
- Manage data with Collections: List, Map, and HashMap
- Apply the Singleton pattern to a service class
- Wire JavaFX ActionEvent, KeyEvent, and MouseEvent handlers
- Connect a working GUI to your own data model

Scenario

You have been given a partially-built JavaFX application called

You have been given a partially-built JavaFX application called **SkyDesk** - a flight booking and seat management system for a fictional airline. The GUI shell already exists however, almost all business logic is missing.

Your task is to work through three stages, building the data model, service layer, and finally connecting everything to the live GUI. At each stage the application becomes more functional. By the end, passengers can search flights, pick a seat on a visual seat map, and manage their bookings - all powered by the classes you wrote.

Project File Map

Unzip the provided starter project. Open it in your IDE and locate the following files:

File	Location	Your Task
`Payable.java`	`model/`	*DO NOT MODIFY - interface to implement*
`Seat.java`	`model/`	*DO NOT MODIFY - abstract base class*
`EconomySeat.java`	`model/`	STAGE 1 - extend Seat
`BusinessSeat.java`	`model/`	STAGE 1 - extend Seat
`FirstClassSeat.java`	`model/`	STAGE 1 - extend Seat

`Passenger.java`	`model/`	STAGE 1 - implement Payable, manage bookings
`Booking.java`	`model/`	STAGE 1 - implement Payable, status logic
`Flight.java`	`model/`	STAGE 1 - build seat map, availability queries
`BookingService.java`	`service/`	STAGE 2 - Singleton, all CRUD logic
`FlightSearchPanel.java`	`ui/`	STAGE 3 - wire search ActionEvents
`SeatMapPanel.java`	`ui/`	STAGE 3 - render seat grid, booking dialog
`PassengerPanel.java`	`ui/`	STAGE 3 - wire passenger management events
`MainApp.java`	`ui/`	*DO NOT MODIFY - provided GUI entry point*

⚠ Before you start

Check that the project compiles and MainApp launches a blank 3-tab window.
Most buttons will show a 'TODO' label - this is expected.
Set your VM arguments: --module-path <javafx-sdk>/lib --add-modules javafx.controls

Stage 1 — Data Model

model/ package • Seat hierarchy • Passenger • Booking • Flight

In Stage 1 you build every class in the `model/` package. The GUI will not change yet - focus entirely on getting the logic right and testing it in isolation.

Task 1.1 - Seat Subclasses

Inheritance **Polymorphism** **Encapsulation**

Open

Open `Seat.java` and read it carefully. It is an **abstract class** - you must not modify it. Your job is to create three concrete subclasses.

➖ Task 1.1a - Create EconomySeat, BusinessSeat, FirstClassSeat

Each class must extend Seat and implement all three abstract methods:

- getBasePrice() - return the base fare (see pricing guide below)
- getSeatClass() - return a string label: "Economy", "Business", or "First Class"
- getPerks() - return a comma-separated string of included perks

Also complete the two concrete methods defined in Seat:

- occupy(String name) - assign a passenger; throw IllegalStateException if already occupied
- vacate() - free the seat and clear passengerName

Override toString() to produce a readable summary (see Javadoc in Seat.java).

ℹ Pricing & Perks Guide

- Economy $99.00 - 1 carry-on, complimentary snack
- Business $349.00 - 2 checked bags, hot meal, priority boarding, lounge access
- First Class $799.00 - 3 checked bags, gourmet dining, flat bed, private suite, chauffeur

➖ Task 1.1b - Encapsulation check

Add at least one additional field to FirstClassSeat (e.g. boolean privateSuite).
Make it private. Provide a getter and a setter.
The setter must validate the input if appropriate - add a comment explaining your choice.

Reflection 1.1

Answer the following in the space provided.

1. Why is Seat declared abstract rather than being a normal class with default method bodies?

Your answer:

2. If you write Seat s = new BusinessSeat("3A"); what type does s.getSeatClass() return at runtime? What OOP principle does this demonstrate?

Your answer:

Task 1.2 - Passenger

Encapsulation | **Interfaces** | **Collections - List**

Open `Passenger.java`. This class must implement the `Payable` interface.

Task 1.2 - Complete Passenger.java

3. Constructor - assign all four fields using this.
4. addBooking(Booking b) - add to the bookings list; throw IllegalArgumentException if b is null.
5. cancelBooking(String id) - remove by bookingId; return true if found and removed, false otherwise.
6. getBookings() - return Collections.unmodifiableList(bookings) so callers cannot modify the list directly.
7. calculateTotal() - sum calculateTotal() across every booking in the list.
8. getReceipt() - build a multi-line String showing every booking and a grand total (see Javadoc for format).
9. Add setters for fullName and email. The email setter must check the value contains '@'; throw IllegalArgumentException if not.
10. Override toString() - e.g. "Jane Doe (PAX-001) - 2 booking(s)".

Hint - calculateTotal()

You can use a simple for-each loop, or Java streams:

```
return bookings.stream()
       .mapToDouble(Payable::calculateTotal)
       .sum();
```

Reflection 1.2

11. What advantage does returning Collections.unmodifiableList() give over returning the raw list?

Your answer:

Task 1.3 - Booking

Interfaces	Encapsulation	Enums

Open `Booking.java`. It links a `Passenger` to a `Seat` on a `Flight`.

➖ Task 1.3 - Complete Booking.java

12. Implement calculateTotal() - return the seat's base price.
13. Implement getReceipt() - single line, e.g. "Booking #BK001 | LHR -> JFK | Economy [12A] | $99.00 | CONFIRMED".
14. Implement cancel() - set status to CANCELLED and call seat.vacate(). Throw IllegalStateException if already cancelled.
15. Implement checkIn() - set status to CHECKED_IN. Throw IllegalStateException if status is not CONFIRMED.

★ Extension - Early-bird discount

In calculateTotal(), apply a 10% discount if the booking is made more than 60 days before departure.
Use ChronoUnit.DAYS.between(bookingTime.toLocalDate(), flight.getDepartureTime().toLocalDate()) to calculate the gap.

Reflection 1.3

16. Why is it better to use an enum (BookingStatus) rather than String constants like "CONFIRMED", "CANCELLED"?

Your answer:

Task 1.4 - Flight

Collections - Map	Polymorphism	Encapsulation

Open `Flight.java`. The seat map is a `Map<String, Seat>` where the key is the seat number (e.g. "12C") and the value is a `Seat` subclass instance.

➖ Task 1.4 - Complete Flight.java

17. initialiseSeatMap() - populate seatMap using the layout below.
18. getSeat(String number) - look up and return the seat, or null if not found.
19. getAvailableSeats() - return a List of all unoccupied seats.
20. getAvailableSeatsByClass(String cls) - filter getAvailableSeats() by getSeatClass().
21. getAvailabilityByClass() - return Map<String, Long> of class name to count of available seats. Use Collectors.groupingBy + Collectors.counting().
22. hasAvailability() - return true if at least one seat is available.
23. Override toString() to produce e.g. "BA0117 | LHR -> JFK | Dep: 2025-06-01 10:30".

ℹ Seat Layout

- Rows 1–2 columns A B C D → FirstClassSeat (8 seats total)
- Rows 3–5 columns A B C D → BusinessSeat (12 seats total)
- Rows 6–15 columns A B C D E F → EconomySeat (60 seats total)

Seat number = row number + column letter, e.g. "1A", "6F", "15C".

```
// Hint - nested loop approach:
String[] fcCols  = {"A","B","C","D"};
for (int row = 1; row <= 2; row++) {
    for (String col : fcCols) {
        String num = row + col;          // e.g. "1A"
        seatMap.put(num, new FirstClassSeat(num));
    }
}
// TODO: repeat for Business (rows 3-5) and Economy (rows 6-15, cols A-F)
```

Reflection 1.4

24. The seatMap stores Seat references, not EconomySeat / BusinessSeat / FirstClassSeat. Why is this a good design decision?

Your answer:

25. What is the difference between getAvailableSeats() and getAvailableSeatsByClass()? Could one be implemented using the other?

Your answer:

Stage 2 — Service Layer

service/ package • Singleton • CRUD • Maps & Collections

Open `BookingService.java`. This is the single point of contact between the GUI and your data model. It stores all flights, passengers, and bookings in `HashMap`s and exposes clean methods for every operation the GUI needs.

Singleton Pattern **Collections - Map** **Encapsulation**

Task 2.1 - Singleton Pattern

Implement getInstance() so that only one BookingService can ever exist.

```
public static BookingService getInstance() {
    if (instance == null) {
        instance = new BookingService();
    }
    return instance;
}
```

Explain in the reflection below why the constructor is private.

Task 2.2 - Flight operations

26. getAllFlights() - return a new ArrayList of flights.values().
27. getFlightByNumber(String num) - return flights.get(num).
28. searchFlights(String origin, String dest) - filter flights by both fields (case-insensitive). Return an empty list if none match.

Task 2.3 - Passenger operations

29. getAllPassengers() - return a new ArrayList of passengers.values().
30. getPassengerById(String id) - return passengers.get(id).
31. registerPassenger(name, email, passport) - auto-generate an ID in the format PAX-003, PAX-004 … using String.format("%03d", passengers.size()+1). Create a new Passenger, add to the map, and return it.

Task 2.4 - Booking operations

32. createBooking(passengerId, flightNumber, seatNumber):

- Look up the Passenger and Flight from their maps; throw IllegalArgumentException if either is not found.
- Call flight.getSeat(seatNumber); throw IllegalArgumentException if the seat is not found.
- Check seat.isOccupied(); throw IllegalStateException if taken.
- Call seat.occupy(passenger.getFullName()).
- Generate bookingId: "BK" + String.format("%03d", bookingCounter++).
- Create a Booking, add to the bookings map, call passenger.addBooking(), return the Booking.

33. cancelBooking(String id) - find the booking, call booking.cancel(), return true/false.
34. getBookingsForPassenger(String id) - filter bookings.values() by passenger ID.
35. getAllBookings() - return a new ArrayList of bookings.values().

➖ Task 2.5 - Expand seed data

In seedData(), add at least 2 more flights (different routes) and 3 more passengers. Create at least 2 sample bookings so the GUI has real data to display immediately.

Reflection 2

36. What would happen if getInstance() were not synchronised in a multi-threaded application? (You do not need to fix this - just explain the risk.)

Your answer:

37. You used HashMap throughout. Name one scenario where a different Map implementation (e.g. TreeMap or LinkedHashMap) would be preferable, and why.

Your answer:

Stage 3 — GUI Integration

ui/ package • ActionEvent • MouseEvent • TableView • Dialog

Now that your model and service are working, wire the live GUI. Each panel already has its layout built - your job is to fill in the TODO event handlers and connect them to BookingService.

Task 3.1 - FlightSearchPanel

ActionEvent **TableView** **Lambda Expressions**

➖ Task 3.1 - Wire the Search tab

38. Search button (LAB TASK 1 in the file)
- Read originField and destField.
- Call service.searchFlights(origin, dest) and assign the result to a List<Flight>.
- Set resultsTable.setItems(FXCollections.observableArrayList(results)).
- Update statusLabel: "Found N flight(s)" or "No flights found."

39. Show All Flights button (LAB TASK 2)
- Call service.getAllFlights() and populate the table the same way.

40. Availability column (LAB TASK 2 cell factory)
- For each Flight cd.getValue(), call getAvailabilityByClass().
- Format the result as "F:6 B:10 E:48" (first letter of each class).

```
// Availability cell value factory example:
avCol.setCellValueFactory(cd -> {
    Map<String,Long> av = cd.getValue().getAvailabilityByClass();
    String txt = "F:" + av.getOrDefault("First Class",0L)
               + "  B:" + av.getOrDefault("Business",0L)
               + "  E:" + av.getOrDefault("Economy",0L);
    return new SimpleStringProperty(txt);
});
```

Task 3.2 - SeatMapPanel

MouseEvent **Polymorphism** **JavaFX Dialog**

➖ Task 3.2a - renderSeatMap() (LAB TASK 1)

Clear seatGrid.getChildren() then add column header Labels (A B C [gap] D E F) in row 0.
Loop over all seats in flight.getSeatMap().values().
For each seat, create a Button with the seat number as text.
Set its background colour based on seat class and occupied status (use the COLOR_* constants already in the file).

```
String fill = seat.isOccupied() ? COLOR_OCCUPIED
    : switch (seat.getSeatClass()) {
        case "First Class" -> COLOR_FIRST;
```

```
            case "Business"    -> COLOR_BUSINESS;
            default            -> COLOR_ECONOMY;
        };
button.setStyle("-fx-background-color:" + fill + ";");
```

Parse the row and column from the seat number (e.g. "12C" -> row 12, col C) to place the button at the correct GridPane position.
If the seat is available, call setOnAction(e -> handleSeatClick(flight, seat)).
If occupied, call button.setDisable(true).

Task 3.2b - buildBookingDialog() (LAB TASK 3)

Create a Dialog<String> (or use a custom Stage).
Show the flight number, seat number, class, and price.
Include a ComboBox<Passenger> populated with service.getAllPassengers().
On Confirm: return the selected passenger's ID. On Cancel: return null.

Task 3.2c - handleSeatClick() (LAB TASK 2)

Call buildBookingDialog(). If the result is non-null:

- Call service.createBooking(passengerId, flight.getFlightNumber(), seat.getSeatNumber()).
- On success: show a confirmation Alert and call renderSeatMap(flight) to refresh colours.
- On exception: show an error Alert with the exception message.

Task 3.3 - PassengerPanel

ActionEvent **TableView** **Collections**

Task 3.3 - Wire Passenger tab

41. refreshPassengerList() - call service.getAllPassengers() and set passengerList.setItems(...).
42. Register button - call service.registerPassenger(), clear form fields, call refreshPassengerList(), show status message.
43. Passenger selection listener - call service.getBookingsForPassenger(...) and set bookingTable.setItems(...).
44. All six bookingTable column cell value factories - replace the TODO SimpleStringProperty placeholders with real data from each Booking.
45. Cancel Booking button - show a confirmation Alert; on OK call service.cancelBooking(...), refresh the table.
46. Print Receipt button - retrieve passenger.getReceipt() and display it in a scrollable TextArea inside a Dialog.

Task 3.4 - Cross-tab Communication

Event-driven design **Callbacks**

When a user double-clicks a flight row in the Search tab, the application should automatically switch to the Seat Map tab and load that flight.

➡ Task 3.4 - Connect Search -> Seat Map

In MainApp.java (read it first - this is the one file you should only extend carefully), notice that searchTab and seatTab are created with their panels.
One approach: pass a Runnable or Consumer<Flight> callback into FlightSearchPanel that triggers tabPane.getSelectionModel().select(seatTab) and seatMapPanel.loadFlight(flight).
Implement loadFlight(Flight f) in SeatMapPanel (the stub is already there).
Implement refreshFlights() and call it when the Seat Map tab is first shown (use tabPane.getSelectionModel().selectedItemProperty() listener).

Reflection 3

47. You used lambda expressions throughout Stage 3. Rewrite one of your lambdas as an anonymous inner class. Which style do you prefer and why?

Rewritten anonymous class + your preference:

48. Describe one way the GUI could become inconsistent if two panels hold separate references to the data instead of sharing a single BookingService instance. How does the Singleton pattern prevent this?

Your answer:

Extension Challenges

Complete these if you finish the main stages. Each one is independent.

★ Extension A - Waitlist Queue

Add WAITLISTED to the BookingStatus enum.
Add a Map<String, Queue<Passenger>> in BookingService keyed by flightNumber.
If createBooking() is called on a full flight, add the passenger to the waitlist queue instead of throwing.
In cancelBooking(), after freeing the seat, automatically confirm the first passenger in the queue if one exists.

★ Extension B - Departure Date Filter

Add a DatePicker to FlightSearchPanel.
Update searchFlights() (or add an overload) to also filter by departure date.
Display departure/arrival times using DateTimeFormatter (e.g. "01 Jun 2025 10:30") in the table columns.

★ Extension C - First Class Private Suite

Use the privateSuite field you added to FirstClassSeat in Task 1.1b.
If privateSuite is true, add a $200 surcharge to getBasePrice().
In renderSeatMap(), use a distinct colour for suite seats.
In buildBookingDialog(), show a checkbox letting the passenger opt for a suite if one is available.

★ Extension D - CSV Persistence

On application exit (Stage.setOnCloseRequest), write all bookings to bookings.csv.
On startup in seedData(), check if the file exists and reload any saved bookings.
Each row: bookingId,passengerId,flightNumber,seatNumber,status